

Chapitre 4

Les Objets Noyaux

en

JavaScript

Table des matières

<u>1.</u>	<u>Objets du noyau</u>	2
<u>a.</u>	<u>L'objet Array</u>	2
<u>b.</u>	<u>L'objet String</u>	6
<u>c.</u>	<u>L'objet Math</u>	8
<u>d.</u>	<u>L'objet Date</u>	10
<u>e.</u>	<u>L'objet Boolean</u>	12

1. Objets du noyau

Les objets **noyau** sont des objets prédéfinis fournis par JavaScript qui ne sont pas reliés aux fenêtres ou aux documents en cours et ils n'ont pas de balises HTML équivalentes. Ils proposent des propriétés et des méthodes permettant d'effectuer facilement un grand nombre de manipulations.

Les objets du noyau JavaScript sont : **Array, Boolean, Date, Function, Math, Number, RegExp et String.**

Le tableau suivant résume les descriptions des objets noyau JS

Objet	Description
Array	Permet de manipuler les tableaux.
String	Permet de manipuler les chaînes de caractères.
Math	Fournit des méthodes pour traiter les fonctions mathématiques usuelles, telles que <i>log</i> , <i>exp</i> , aussi bien que les fonctions trigonométriques.
Date	Permet de travailler avec la date courante; fournit également des méthodes pour faire des opérations sur les dates, les heures, les minutes et les secondes.
Boolean	Permet de manipuler les valeurs logiques.
Number	Permet de manipuler les valeurs numériques et les constantes.
Function	Permet d'emballer un code dans une fonction ¹ .
RegExp	Permet de créer un objet expression rationnelle pour la reconnaissance d'un modèle dans un texte.

Remarque :

Il ya d'autres objets JS prédéfinis (Error, Symbol,...) que vous pouvez étudier en consultant l'adresse URL : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Objets_globaux

a. L'objet Array

La notion de tableau en JavaScript est différente de celle d'un autre langage, car JavaScript est faiblement typé. En effet, les éléments d'un tableau peuvent avoir des types différents (string, nombre, ...) puisqu'il considéré comme objet.

Pour chaque tableau, des propriétés sont lues ou écrites, des méthodes sont appliquées.

i. Création de tableau :

Dans JavaScript, les tableaux sont créés à partir de l'objet prédéfini Array.

- **Créer un tableau de taille indéfinie :**

Tout nouveau tableau doit être déclaré, suivant la syntaxe: `nomTableau = new Array();`

¹ Les fonctions en tant qu'objets ont été citées dans la rubrique Function comme Objet dans le chapitre 3

Il n'est pas nécessaire de préciser le nombre d'éléments du tableau, sa dimension. Celle-ci sera automatiquement ajustée par JavaScript en fonction du contenu du tableau. Autrement dit, la taille d'un tableau change, de façon dynamique, au fur et à mesure qu'on lui affecte des valeurs. Elle est définie par le plus gros indice utilisé.

Noter que pour accéder à un élément du tableau, on utilise les accolades []:

Par exemple : `var tab = new Array();`

`tab[39] = "dernier element";` // Ce code crée un tableau avec 40 éléments.

- **Créer un tableau de taille fixe :**

Il est possible aussi de fixer la taille d'un tableau lors de sa création :

`LeTableau=new Array(dimension);` // où la variable *dimension* précise le nombre d'éléments

Exemple : `var tab = new Array(40);`

- **Associer des valeurs à tous ses éléments au moment de la création d'un tableau**

Il est possible d'affecter directement un contenu à chaque cellule du tableau lors de la déclaration.

Par exemple : `var tab = new Array("1er Element", 2, "3e Element");`

Le tableau tab a trois éléments avec les valeurs :

`tab[0] = "1er Element" ;` // Attention le 1er élément à pour indice 0!

`tab[1] = 2`

`tab[2] = "3e Element" ;` //Le dernier a pour indice (dimension-1)

ii. Propriétés de l'objet Array :

L'objet Array possède deux propriétés caractéristiques: les propriétés *input* et *length*.

Le tableau suivant décrit les propriétés de l'objet Array.

Propriété	Description
length	Cette propriété contient le nombre d'éléments du tableau.
input	Cette propriété permet de faire une recherche dans le tableau à l'aide d'une expression régulière

iii. Méthodes standards de l'objet Array

Le tableau suivant décrit les méthodes de l'objet Array

➤ L'objet Array

On considère la déclaration des tableaux t1, t2 et t3 comme suit :
`let t1 = ['a', 'b', 'c', 'd'];`
`let t2 = [1,2];` `let t3 = [17,28, 53, 107, 89, 14, 220];`

Méthode et syntaxe	Description	Exemple	Résultat
nomTab.concat(tab1, tab2, ...[tabi])	Concatène plusieurs tableaux et retourne le tableau obtenu	alert(t1.concat(t2)) ;	a, b , c, d, 1, 2
nomTab.filter(fonctionCallback)	Renvoie un nouveau tableau contenant tous les éléments du tableau d'origine qui remplissent une condition déterminée par la fonction callback	alert (t3.filter(e=> e<60)); function test(x) {return x%2==0 && x >20} alert (t3.filter(test));	17, 28, 53, 14 28, 220
nomTab.find(fonctionCallback)	Renvoie la valeur du premier élément trouvé dans le tableau qui respecte la condition donnée par la fonction de test passée en argument. Sinon, la valeur <i>undefined</i> est renvoyée.	alert(t3.find(elt=> elt%2==0));	28
nomTab.forEach(fctCallback)	Exécute une fonction donnée sur chaque élément du tableau	t2.forEach(elt => document.write(elt+ "-")); let t=[]; t2.forEach (x => t.push(x+2)); alert(t);	1-2- 3, 4
nomTab.join() ; nomTab.join ('séparateur') ;	Concatène tous les éléments d'un tableau et les retourne dans une chaîne de caractères . La concaténation utilise la virgule ou une autre chaîne, fournie en argument, comme séparateur.	alert(t1.join()) ; alert(t1.join ('*'));	a, b, c, d a*b*c*d
nomTab.map(fonctionCallback)	Renvoie un	alert(t2.map(elt=>	5, 10

	nouveau tableau avec les résultats de l'appel d'une fonction fournie sur chaque élément du tableau appelant.	elt*5));	
nomTab.pop() ;	Supprime le dernier élément d'un tableau et retourne cet élément	alert(t1. pop ()) ; alert(t1) ;	d a, b, c
nomTab.push() ;	Ajoute un ou plusieurs éléments à la fin d'un tableau et retourne la nouvelle taille du tableau.	alert(t1. push ('e')) ; alert(t1) ;	5 a, b, c, d , e
nomTab.reverse() ;	Inverse l'ordre des éléments du tableau (le tableau est modifié)	alert(t1. reverse ())	d, c, b, a
nomTab.shift() ;	Retire le premier élément d'un tableau et renvoie cet élément.	alert(t1. shift ()) ; alert(t1) ;	a b, c, d
nomTab.unshift(e1, el2,...) ;	Ajoute un ou plusieurs éléments au début d'un tableau et renvoie la nouvelle longueur du tableau	alert(t1. unshift ('e','f')) ; alert(t1);	6 e, f, a, b, c, d
nomTab.slice(indiceDéb) ; nomTab.slice(indiceDéb, indiceFin) ;	Renvoie un tableau , contenant une portion du tableau d'origine délimité par : un indice de début et un indice de fin (exclus) : [indDéb, indFin[.	alert(t1. slice (2)); alert(t1. slice (1,4));	c, d b, c, d
nomTab.splice(index, nbEl, el1,el2,...) ;	Supprime nbEl éléments à partir de la position	t1. splice (1,1,'x','y'); alert(t1) ;	a, x, y, c, d

	index et ajoute les éléments el1, el2,....		
nomTab.sort(); nomTab.sort(fctComparaison);	Trie les éléments du tableau dans l'ordre croissant (selon le code UTF-16 après conversion en chaîne de caractères) S'il y a une fonction de comparaison fctComp(a,b), le tri se fait selon la valeur de retour de cette fonction : < 0 : l'élément a est placé avant b = 0 : a et b sont inchangés > 0 : a est placé après b	t1.splice(1,0,'x','t'); alert(t1); alert(t1.sort()); alert(t3.sort((a,b)=>a-b));	a, x, t, b, c, d a, b, c, d, t, x 14, 17, 28, 53, 89, 107, 220
nomTab.toString();	Renvoie une chaîne de caractères représentant le tableau spécifié et ses éléments.	alert(t1.toString());	a, b, c, d
nomTab.valueOf();	Renvoie la valeur primitive du tableau	alert(t1.valueOf());	a, b, c, d

b. L'objet String

Pour manipuler les chaînes de caractères, le langage JavaScript propose l'objet String

On signale dans la littérature une limitation de la longueur des strings à 50/80 caractères. Cette limitation du compilateur JavaScript peut toujours être contournée par l'emploi de signes + et la concaténation.

1. Les propriétés de l'objet String

L'objet *string* a une seule propriété : la propriété **length** qui permet de retourner la longueur d'une chaîne de caractères. Exemple: x = chaine.length; x = ('chaine de caracteres').length;

iv. Les méthodes de l'objet String

Les méthodes de l'objet *string* permettent de récupérer une portion d'une chaîne de caractère, ou bien de la modifier. Le tableau suivant décrit les méthodes de l'objet *String* :

On considère la déclaration des 2 chaînes ch1 et ch2 comme suit : **let ch1= 'iSet';**

let ch2= 'charGuia'; let ch3="ceci est un test";

Méthode et syntaxe	Description	Exemple	Résultat
Chaine.anchor(ch) ;	Crée une ancre HTML dont l'attribut name a pour valeur l'argument passé en paramètre	alert(ch1. anchor ('nom'));	 iSet
Chaine.charAt(pos) ;	Renvoie le caractère à la position passée en argument	alert(ch1. charAt (3));	t
Chaine.charCodeAt(pos) ;	Renvoie le code UTF-16 du caractère situé à la position donnée	alert(ch1. charCodeAt (2));	101
Chaine.concat(ch1, ch2, ch1,...) ;	Concatène avec les chaînes passées en paramètre	alert (ch1. concat ('*', ' ', ch2));	iSet*charGuia
Chaine.endsWith(ch);	Renvoie un booléen indiquant si la chaîne de caractères se termine par la chaîne de caractères fournie en argument.	alert(ch2. endsWith ('ia'));	true
String.fromCharCode(nb) ;	Renvoie une chaîne de caractères créée à partir de points de code UTF-16.	Let caractere = String.fromCharCode(65);	A
Chaine.indexOf(sousChaîne, pos) ;	Renvoie l' indice de la première occurrence de la chaîne cherchée dans la chaîne courante (à partir de pos). Elle renvoie -1 si la valeur cherchée n'est pas trouvée.	alert(ch2. indexOf ('a')); alert(ch2. indexOf ('a',3));	2 7
Chaine.lastIndexOf(sous Chaîne, pos) ;	Même principe que indexOf, mais, avec un parcours de droite à gauche.	alert(ch2. lastIndexOf ('a')); alert(ch2. lastIndexOf ('a',4));	7 2
Chaine.link(Url) ;	Transforme la chaîne en lien hypertexte vers l'URL passée en paramètre	alert(ch1. link ("www.iset.tn"));	 iSet
Chaine.split(séparateur) ;	Divise une chaîne de caractères à partir d'un séparateur pour fournir un tableau de sous-chaînes	let tab = ch3. split (" "); alert(tab+ " * "+tab.length);	ceci,est,un,t est * 4
Chaine.substring(pos1, pos2) ;	Renvoie une sous-chaîne de la chaîne courante, entre un indice de début et un indice de fin.	alert(ch2. substring (2,5));	arG

Chaine.toLowerCase() ;	Renvoie la chaîne de caractères courante en minuscules (sans modifier la chaîne initiale)	alert(ch1. toLowerCase());	iset
Chaine.toUpperCase() ;	renvoie la chaîne de caractères courante en majuscules (sans modifier la chaîne initiale)	alert(ch1. toUpperCase());	ISet
Chaine.trim() ;	Retire les blancs (espace, tabulation, espace insécable, caractères de fin de ligne (LF, CR, etc..) en début et fin de chaîne.	let ch=' iset '; let cht = ch.trim(); alert(cht. trim() + cht.length);	iset4
Chaine.valueOf() ;	Renvoie la valeur primitive de l'objet String	alert(ch1. valueOf());	iSet

c. L'objet Math

L'objet Math est, un objet qui a de nombreuses méthodes et propriétés permettant de manipuler des nombres et qui contient des fonctions mathématiques courantes. Par exemple Math.cos(1);

1. Les méthodes et propriétés standards de l'objet Math

Le tableau suivant décrit les méthodes de l'objet *Math*.

Méthode	Description	Exemple
abs()	renvoie la valeur absolue d'un nombre, il renvoie donc le nombre s'il est positif, son opposé (positif) s'il est négatif	• x = Math.abs(3.26); //donne x = 3.26 • x = Math.abs(-3.26); //donne x = 3.26
ceil()	Renvoie le plus petit entier supérieur ou égal à la valeur donnée en paramètre	• x = Math.ceil(6.01); //donne x = 7 • x = Math.ceil(3.99); //donne x = 4
floor()	retourne le plus grand entier inférieur ou égal à la valeur donnée en paramètre.	• x = Math.floor(6.01); //donne x = 6 • x = Math.floor(3.99); //donne x = 3
round()	Arrondit à l'entier le plus proche la valeur donnée en paramètre. Si la partie décimale de la valeur entrée en paramètre vaut 0.5, la méthode <i>Math()</i> arrondi à l'entier supérieur.	• x = Math.round(6.01); //donne x = 6 • x = Math.round(3.80); //donne x = 4 • x = Math.round(3.50); //donne x = 4
max(Nombre1, Nombre2)	renvoie le plus grand des deux nombres donnés en paramètre	• var x = Math.max(6,7.25); //donne x = 7.25 • var x = Math.max(-8.21,-3.65);

		//donne x = -3.65 • <code>var x = Math.max(5,5);</code> //donne x = 5
min(Nombre1, Nombre2)	Retourne le plus petit des deux nombres donnés en paramètre	• <code>x = Math.min(6,7.25);</code> //donne x = 6 • <code>x = Math.min(5,5);</code> //donne x = 5
pow(Valeur1, Valeur2)	Retourne le nombre <i>Valeur1</i> à la puissance <i>Valeur2</i>	• <code>x = Math.pow(3,3);</code> //donne x = 27
random()	La méthode <i>random()</i> renvoie un nombre pseudo-aléatoire compris entre 0 et 1. La valeur est générée à partir des données de l'horloge de l'ordinateur.	• <code>x = Math.random();</code> //donne x = 0.6489534931546957
sqrt(Valeur)	Renvoie la racine carrée du nombre passé en paramètre	• <code>x = Math.sqrt(9);</code> //donne x = 3

2. Méthodes Logarithmes et exponentielle

Méthode	Description
Math.E	Propriété qui retourne le nombre d'Euler (environ 2.718).
Math.exp(valeur)	Cette méthode renvoie l'exponentielle de la valeur entrée en paramètre.
Math.LN2	La propriété <i>LN2</i> fournit le logarithme népérien de 2.
Math.LN10	Propriété donne le logarithme népérien de 10.
Math.log(valeur)	La méthode <i>log()</i> renvoie le logarithme de la valeur entrée en paramètre.
Math.LOG2E	Propriété qui renvoie la valeur du logarithme du nombre d'Euler en base 2.
Math.SQRT1_2	Propriété qui retourne la valeur de "1 divisé par racine de 2" (0.707).
Math.SQRT2	La propriété <i>SQRT2</i> (<i>Square Root 2</i>) donne la racine de 2 (1.414).

3. Trigonométrie

Méthode	Description
Math.PI	Retourne la valeur du nombre PI, soit environ 3.1415927
Math.sin(valeur)	Retourne le sinus de la valeur entrée en paramètre (doit être donnée en radians). La valeur retournée est comprise dans l'intervalle [-1;1].
Math.asin(valeur)	Retourne l'arcsinus de la valeur entrée en paramètre. La valeur doit être comprise dans l'intervalle [-1;1]. Dans le cas contraire, la méthode <i>asin()</i> renvoie la valeur <i>NaN</i> (<i>Not a Number</i>).
Math.cos(valeur)	Retourne le cosinus de la valeur entrée en paramètre (doit être donnée en radians). La valeur retournée est comprise dans l'intervalle [-1;1].

Math.acos(valeur)	Retourne l'arccosinus de la valeur entrée en paramètre. La valeur doit être comprise dans l'intervalle [-1;1]. Dans le cas contraire, la méthode <i>acos()</i> renvoie la valeur <i>NaN (Not a Number)</i> .
Math.tan(valeur)	Retourne la tangente de la valeur entrée en paramètre (doit être donnée en radians)
Math.atan(valeur)	Retourne l'arctangente de la valeur entrée en paramètre. La valeur doit être comprise dans l'intervalle [-1;1]. Dans le cas contraire, la méthode <i>atan()</i> renvoie la valeur <i>NaN (Not a Number)</i> .

d. L'objet *Date*

L'objet *Date* permet de travailler avec toutes les variables qui concernent les dates et la gestion du temps. Il s'agit d'un objet inclus de façon native dans JavaScript, et que l'on peut toujours utiliser.

1. Syntaxe de l'objet date

La syntaxe pour créer un objet-date peut être une des suivantes :

1. **Nom_de_l_objet = new Date()**
cette syntaxe permet de stocker la date et l'heure actuelle
2. **Nom_de_l_objet = new Date("jour, mois date année heures:minutes:secondes")**
les paramètres sont une chaîne de caractère suivant scrupuleusement la notation ci-dessus
3. **Nom_de_l_objet = new Date(année, mois, jour)**
*les paramètres sont trois entiers séparés par des virgules.
Les paramètres omis sont mis à zéro par défaut*
4. **Nom_de_l_objet = new Date(année, mois, jour, heures, minutes, secondes[, millisecondes])**
*les paramètres sont six entiers séparés par des virgules.
Les paramètres omis sont mis à zéro par défaut*

Les dates en JS sont le nombre de millisecondes depuis le 1^{er} janvier 1970. Ainsi, toute date antérieure au 1^{er} janvier 1970 fournira une valeur erronée.

A partir de la version 1.3, il est possible de manipuler des dates de plus ou moins 100 000 000 de jours par rapport au premier janvier 1970.

2. Les méthodes de l'objet *Date*

La date est stockée dans une variable sous la forme d'une chaîne qui contient le jour, le mois, l'année, l'heure, les minutes, et les secondes.

Syntaxe :

Objet_Date.Methode()

Connaître la date

Les méthodes dont le nom commence par le radical **get** permettent de renvoyer une partie d'un objet Date

On considère l'objet **d** défini comme suit : **let d = new Date(2019,2,11, 10,15,47);**
d.setMilliseconds(123);

Soit le **Lundi 11 Mars 2019 10h15 47sec 123 millisecondes**

Méthode et syntaxe	Description	Exemple	Résultat
d.getDate() ;	Renvoie le jour du mois (entre 1 et 31)	alert(d.getDate());	11
d.getDay() ;	Renvoie le jour de la semaine (entre 0 et 6 à commencer à partir de dimanche)	alert(d.getDay());	1
d.getFullYear() ;	Renvoie l' année sur 4 chiffres	alert(d.getFullYear());	2019
d.getHours();	Renvoie l' heure actuelle (entre 0 et 23).	alert(d.getHours());	10
d.getMilliseconds();	Renvoie le millième de seconde (entier >0)	alert(d.getMilliseconds());	123
d.getMinutes();	Renvoie le nombre de minutes (entre 0 et 59)	alert(d.getMinutes());	15
d.getMonth() ;	Renvoie le mois (entre 0 et 11)	alert(d.getMonth());	2
d.getSeconds() ;	Renvoie le nombre de secondes (entre 0 et 59)	alert(d.getSeconds());	47
d.getTime() ;	Renvoie le nombre de millisecondes depuis le 1 ^{er} Janvier 1970 (entier)	alert(d.getTime());	1552295747123
d.getTimezoneOffset() ;	Renvoie la différence en minutes entre le fuseau horaire GMT (Greenwich Mean Time) et celui de la date d	alert(d.getTimezoneOffset());	-60
d.toString() ;	Renvoie dans une chaîne de caractères la date traduite en anglais	alert(d.toString());	Mon Mar 11 2019
d.toLocaleString() ;	Renvoie une chaîne de caractères représentant la date	alert(d.toLocaleString());	11/03/2019 à 10:15:47

	selon le format local		
--	-----------------------	--	--

Modifier le format de la date

Les deux méthodes suivantes ne permettent de travailler que sur l'heure actuelle (objet *Date()*) leur syntaxe est donc figée :

Méthode	Description	Type de valeurs retournée
toGMTString()	Permet de convertir une date en une chaîne de caractères au format GMT	L'objet retourné est une chaîne de caractère du type : Wed, 28 Jul 1999 15:15:20 GMT
toLocaleString()	Permet de convertir une date en une chaîne de caractères au format local	L'objet retourné est une chaîne de caractère dont la syntaxe dépend du système, exple : 28/07/99 15:15:20

Modifier la date

Les méthodes dont le nom commence par le radical **set** permettent de modifier une valeur :

Méthode	Description	Type de valeur en paramètre
Nomd.setDate(X)	Permet de fixer la valeur du jour du mois	Le paramètre est un entier (entre 1 et 31) qui correspond au jour du mois
Nomd.setDay(X)	Permet de fixer la valeur du jour de la semaine	Le paramètre est un entier qui correspond au jour de la semaine : 0: dimanche 1: lundi ...
Nomd.setHours(X)	Permet de fixer la valeur de l'heure	Le paramètre est un entier (entre 0 et 23) qui correspond à l'heure
Nomd.setMinutes(X)	Permet de fixer la valeur des minutes	Le paramètre est un entier (entre 0 et 59) qui correspond aux minutes
Nomd.setMonth(X)	Permet de fixer le numéro du mois	Le paramètre est un entier (entre 0 et 11) qui correspond au mois : 0: janvier 1: février ...
Nomd.setTime(X)	Permet d'assigner la date	Le paramètre est un entier représentant le nombre de millisecondes depuis le 1 ^{er} janvier 1970

e. L'objet *Boolean*

L'objet Boolean est un objet du noyau JavaScript permettant de créer et de manipuler des valeurs de type booléennes.

Syntaxe :

```
var x = new Boolean(expression);
```

Le paramètre peut-être soit une valeur (*True* ou *False*) soit une expression, auquel cas celle-ci est évaluée en tant que valeur booléenne. Lorsqu'aucune valeur n'est passée, ou la valeur *0*, ou une chaîne de caractères vide, ou *null*, ou *undefined* ou bien *NaN*, la valeur de l'objet est initialisée à *False*. Dans tous les autres cas, l'objet *Boolean* possédera la valeur *True*.

1. Les propriétés de l'objet Boolean

Le tableau suivant décrit les propriétés de l'objet *Boolean*.

Propriété	description
constructor	Cette propriété contient le constructeur l'objet <i>Boolean</i> .
prototype	Cette propriété permet d'ajouter des propriétés personnalisées à l'objet.

v. Les méthodes standards de l'objet Boolean

Le tableau suivant décrit les méthodes de l'objet *Boolean*.

Méthode	description
toSource()	Cette méthode renvoie le code source qui a permis de créer l'objet <i>Boolean</i> .
toString()	Cette méthode renvoie la chaîne de caractères correspond à l'instruction qui a permis de créer l'objet <i>Boolean</i> .
valueOf	Cette méthode retourne tout simplement la valeur de l'objet <i>Boolean</i> auquel elle fait référence.

Références

<https://members.loria.fr/ABelaid/Enseignement/isc-12/cours4-javascript-objet.pdf>

<http://www.trucsweb.com/tutoriels/javascript/tw331/>

<https://www.dark-rider.fr/fr/manipulation-des-formulaires/>

<http://www.siteduzero.com/tutoriel-3-8047-les-formulaires-de-bons-am..>

<https://fr.javascript.info/arrow-functions-basics>

<https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference>

le tableau de synthèse avec les principales méthodes, Amel Triki, 2019